hwrng — Direct RDSEED Hardware RNG for x86_64

A Python C extension that exposes Intel/AMD RDSEED directly — bypassing the OS entropy pool entirely. Raw CPU thermal noise, no CSPRNG conditioning, no buffering.

Platform: x86_64 only. RDSEED is an x86 instruction — ARM (Apple Silicon, Raspberry Pi) is not supported and will fail at build time with a clear error.

What it does

Function	Description
hwrng.has_rdseed()	Returns True if the CPU supports RDSEED
hwrng.rdseed_raw_bytes(n, max_retries=100)	Returns n bytes of raw hardware entropy

n must be a positive multiple of 8 (64-bit words). Maximum 1 MB per call.

```
import hwrng


if not hwrng.has_rdseed():
   raise RuntimeError("CPU does not support RDSEED")


entropy = hwrng.rdseed_raw_bytes(64)
print(entropy.hex())
```

AMD Zen 5 note

Zen 5 CPUs have a known RDSEED bug (Oct 2025) where an exhausted entropy pool returns 0 with the success flag set. This library mitigates it: any zero result is treated as a failed attempt and retried. A UserWarning is raised at import time on affected hardware. Update CPU microcode (Oct 2025 release) to fix at the hardware level.

Requirements

- x86_64 CPU with RDSEED support (Intel Broadwell 2014+, AMD Zen 2019+)
- Python 3.14+
- C compiler with <immintrin.h> support

Building from source

Linux (Ubuntu/Debian)

```
sudo apt install python3-dev ninja-build
pip install meson-python meson
pip install -e . --no-build-isolation
```

Linux (Arch/Manjaro)

```
sudo pacman -S python ninja meson
```

pip install meson-python

pip install -e . --no-build-isolation

macOS (Intel only)

brew install ninja meson python@3.14

pip install meson-python

pip install -e . --no-build-isolation

Apple Silicon (M1/M2/M3/M4) is ARM — build will fail at the RDSEED intrinsic check with a clear error message. This is intentional.

Windows — MSYS2 UCRT64 (GCC build)

This produces a GCC-compiled .pyd linked against UCRT64 Python. Install into a dedicated UCRT64 venv to keep it isolated from any Windows Python installation.

1. Install MSYS2 from https://www.msys2.org and open the **UCRT64** shell.

2. Install system dependencies via pacman:

pacman -S mingw-w64-ucrt-x86_64-python \

    mingw-w64-ucrt-x86_64-gcc \

    mingw-w64-ucrt-x86_64-meson \

    mingw-w64-ucrt-x86_64-ninja

Do **not** use pip install ninja — the ninja PyPI package fails to build under UCRT64 GCC 16 due to an mkdtemp symbol ambiguity. The pacman ninja is correct.

3. Create a dedicated UCRT64 venv:

/ucrt64/bin/python -m venv ~/venv-ucrt64

source ~/venv-ucrt64/bin/activate # note: bin/ not Scripts/ — Linux convention

4. Install the Python build backend:

pip install meson-python meson --no-build-isolation

--no-build-isolation uses pacman's meson+ninja instead of pulling from PyPI

5. Clone and build:

git clone https://github.com/youruser/hwrng.git

cd hwrng

pip install -e . --no-build-isolation

6. Verify the build:

Should show True on supported hardware

python -c "import hwrng; print(hwrng.has_rdseed())"

64 bytes of raw hardware entropy

python -c "import hwrng; print(hwrng.rdseed_raw_bytes(64).hex())"

Confirm it loaded from the UCRT64 venv, not a Windows Python installation

python -c "import hwrng, importlib.util; print(importlib.util.find_spec('hwrng').origin)"

The wheel produced will be named hwrng-1.0.0-cp314-cp314-mingw_x86_64_ucrt_gnu.whl.

Windows — MSVC (Windows Python build)

This produces an MSVC-compiled .pyd linked against the official python.org Python. Use this if you need the extension in a standard Windows Python environment.

Requirements:

- Python 3.14 from https://python.org
- Visual Studio 2022 Build Tools with "Desktop development with C++" workload
- MSYS2 UCRT64 for meson and ninja (or install via pip in a clean venv)

python -m venv venv

venv\Scripts\activate

pip install meson-python meson ninja

pip install -e . --no-build-isolation

meson auto-detects MSVC via cc.get_id() == 'msvc' and applies /arch:AVX2 /O2 flags. No manual compiler selection needed.

Build system internals

The build uses meson-python as the PEP 517 backend. meson.build handles three compiler paths automatically:

Compiler detected	Platform	Flags applied
msvc / clang-cl	Windows MSVC	/arch:AVX2 /O2, links python3.lib
gcc / clang on Darwin	macOS	-mrdseed -msse2 -march=native -O2
gcc / clang elsewhere	Linux, MSYS2	-mrdseed -msse2 -O2

Py_LIMITED_API 0x030e0000 (Python 3.14) is set in all paths — the extension uses the stable ABI and will work with any Python 3.14+ without recompilation.

The build fails fast with a clear error if:

- The CPU/compiler does not support RDSEED intrinsics
- The target is ARM (Apple Silicon or otherwise)

Troubleshooting

error: externally-managed-environment (MSYS2) MSYS2 protects its system Python from pip. Always use a venv:

/ucrt64/bin/python -m venv ~/venv-ucrt64

source ~/venv-ucrt64/bin/activate

Failed building wheel for ninja (MSYS2) Do not install ninja via pip under MSYS2. Install via pacman and use --no-build-isolation so the build picks up /ucrt64/bin/ninja instead.

bash: .../Scripts/activate: No such file or directory (MSYS2) UCRT64 Python creates bin/ not Scripts/. Use source ~/venv-ucrt64/bin/activate.

RDSEED exhausted — hardware entropy depleted The hardware entropy pool was drained. Reduce request size, increase max_retries, or add a small delay between calls. On AMD Zen 5, update CPU microcode.

Wrong Python loaded (Windows venv shadowing UCRT64) If which python shows a path under a Windows venv's Scripts/, that venv is ahead in PATH. Strip it for the session:

```
export PATH=$(echo "$PATH" | tr ':' '\n' | grep -v '/home/juhi/venv/Scripts' | tr '\n' ':')
```

License